



KineTrex

Name:- Rashmi Ranjan Sahoo

Application Id :- KTS020260713319

Gmail :- rashmiranjansahoogodu@gmail.com

Project Name :- Library Management System

ABSTRACT

The Smart Library Management System is a web-based application developed to automate and simplify the daily operations of a library. Traditional library systems that rely on manual record keeping often face challenges such as inefficient book tracking, time-consuming search processes, inaccurate inventory management, misplaced records, and difficulties in monitoring issued and returned books. The proposed system addresses these limitations by providing a secure, user-friendly, and centralized platform for managing library resources efficiently. The application is designed using Python Flask as the backend framework, MongoDB Atlas as the cloud-based NoSQL database, and HTML5, CSS3, and Vanilla JavaScript for creating a responsive and interactive user interface. The system offers an integrated environment where librarians, students, and teachers can access library services according to their respective roles while maintaining data security and operational efficiency.

The system provides a comprehensive set of functionalities required for modern library management. Librarians can securely register and log in to the system, manage book categories, add new books, update existing records, delete unavailable books, and maintain complete inventory details, including ISBN, author, publisher, quantity, category, and shelf location. The system also allows librarians to manage member information by registering students, teachers, and guests, updating their profiles, maintaining membership status, and monitoring borrowing history. A role-based authentication mechanism ensures that administrators and members have access only to the features relevant to their responsibilities. Passwords are securely encrypted using bcrypt hashing, while session management, CSRF protection, and server-side validation enhance the overall security of the application.

One of the major features of the proposed system is the automated Book Issue and Return Management module. Librarians can issue books to registered members, monitor due dates, process book returns, and calculate overdue fines automatically based on predefined library policies. The system eliminates manual calculations by generating fines dynamically according to the number of delayed days, thereby reducing human errors and ensuring transparency in library operations. In addition, complete issue history, overdue tracking, and fine payment records are maintained within the database, enabling efficient monitoring of library activities. The dashboard provides real-time statistical information, including the total number of books, available books, issued books, registered members, and graphical representations of issue statistics and book categories, allowing librarians to make informed decisions regarding library management.

The application also includes a dedicated Student and Teacher Portal, allowing members to log in independently and access personalized library services. Through this portal, users can browse the library catalog, search books by title, author, ISBN, or category, view newly added books, monitor issued books along with due dates, check pending fines, and update their personal profile information. This self-service functionality minimizes the workload of librarians while providing members with convenient access to library resources from any location through a web browser. The responsive user interface ensures compatibility across desktops, laptops, tablets, and mobile devices, thereby improving accessibility and user

experience.

MongoDB Atlas serves as the cloud-based database for storing information related to books, members, issued books, returned books, administrators, categories, settings, activity logs, and fine payment records. The use of a NoSQL database provides flexibility in handling large volumes of structured and semi-structured data while supporting efficient retrieval and scalability. Flask facilitates communication between the user interface and the database through RESTful APIs, enabling reliable Create, Read, Update, and Delete (CRUD) operations. Input validation, secure database interactions, and modular project architecture improve maintainability, scalability, and overall software quality. The project follows industry-standard software engineering practices, ensuring clean code organization and ease of future enhancements.

The Smart Library Management System significantly reduces paperwork, minimizes manual errors, improves record accuracy, and enhances the overall efficiency of library administration. Automated reporting, inventory tracking, member management, and secure authentication simplify the day-to-day responsibilities of librarians while improving service quality for students and teachers. The cloud-based database enables centralized storage and easy access to information, making the system suitable for schools, colleges, universities, and other educational institutions.

In conclusion, the proposed Smart Library Management System provides a secure, reliable, and scalable solution for digital library management by integrating modern web technologies with cloud database services. The project demonstrates the practical implementation of full-stack web development concepts, database management, authentication mechanisms, and responsive user interface design to solve real-world library management challenges. Future enhancements may include barcode and QR code integration, email and SMS notifications for due dates, online book reservation and renewal, mobile application support, multilingual interfaces, advanced analytics, and AI-based book recommendation features to further improve the effectiveness and usability of the system.

Chapter-1 :- Introduction

In the digital era, educational institutions are increasingly adopting information technology to automate their administrative and academic processes. A library is one of the most important resources in any educational institution, providing books, journals, magazines, research materials, and digital resources to students, teachers, and researchers. Traditional library management methods rely heavily on manual record-keeping, paper registers, and physical documentation for maintaining book inventories, issuing and returning books, managing member information, and calculating overdue fines. These manual processes are time-consuming, prone to human error, difficult to maintain, and inefficient when handling large volumes of library data.

The Smart Library Management System is a web-based application designed to automate and simplify the daily operations of a library. The system provides a centralized platform for librarians, students, and teachers to efficiently manage library activities through an intuitive and secure interface. It is developed using Python Flask as the backend framework, MongoDB Atlas as the cloud-based NoSQL database, and HTML5, CSS3, and Vanilla JavaScript for building a responsive and user-friendly frontend.

The system enables librarians to perform essential library operations such as adding, updating, deleting, and searching books, managing categories, maintaining member records, issuing and returning books, tracking overdue books, calculating fines automatically, and generating reports. Students and teachers can access their dedicated portal to search books, view issued books, monitor due dates, check pending fines, and update their personal profiles. Secure authentication, password encryption, session management, input validation, and role-based access control ensure that only authorized users can access system resources.

By digitizing library operations, the proposed system reduces paperwork, minimizes manual errors, improves record accuracy, enhances operational efficiency, and provides faster access to library information. The application also supports real-time statistics, inventory tracking, and cloud-based data storage, making it suitable for schools, colleges, universities, and other educational institutions.

1.1 Purpose

The primary purpose of the Smart Library Management System is to automate the complete library management process and replace traditional manual operations with a secure, efficient, and centralized digital solution. The system is designed to simplify the responsibilities of librarians while providing convenient access to library services for students and teachers.

The specific purposes of the system are:

- To automate book management, including adding, updating, deleting, and searching book records.
- To maintain accurate information about books, authors, publishers, categories, ISBN numbers, and inventory.

- To manage student, teacher, and guest membership records efficiently.
- To simplify the book issue and return process through automated workflows.
- To calculate overdue fines automatically based on predefined library policies.
- To provide secure authentication and role-based access for administrators, students, and teachers.
- To allow library members to search books, view issued books, monitor due dates, and manage their profiles.
- To generate reports related to books, members, issued books, and library activities.
- To reduce paperwork, improve operational efficiency, and minimize human errors.
- To provide a scalable cloud-based solution using MongoDB Atlas for secure and reliable data storage.

1.2 Scope

The scope of the Smart Library Management System covers all major activities involved in the administration and management of a modern library. The system is intended for educational institutions such as schools, colleges, universities, training centers, and departmental libraries.

The project scope includes:

- Administrator registration and secure login.
- Student and teacher portal with separate authentication.
- Book catalog management and inventory tracking.
- Category management for organizing books.
- Member registration and profile management.
- Book issue and return management.
- Automatic overdue fine calculation.
- Book availability checking and search functionality.
- Dashboard displaying real-time library statistics.
- Report generation for books, members, and issued books.
- Activity logging for administrative operations.
- Library configuration and settings management.
- Responsive web interface accessible from desktop, laptop, tablet, and mobile devices.
- Cloud-based database management using MongoDB Atlas.

1.3 Features

The Smart Library Management System provides a comprehensive set of features that improve the efficiency, accuracy, and security of library operations.

Authentication and Security

- Secure administrator registration and login.
- Separate login portal for students and teachers.
- Password encryption using bcrypt.
- Session management for secure access.
- Role-based authentication and authorization.
- Input validation and CSRF protection.

Dashboard

- Real-time statistics of books and members.
- Dynamic charts and graphical reports.
- Quick access to frequently used operations.
- Recent activity monitoring.

Book Management

- Add new books.
- Edit existing book details.
- Delete unavailable books.
- Search books by title, author, ISBN, or category.
- Upload book cover images.
- Manage stock and inventory.

Member Management

- Register students, teachers, and guests.
- Update member information.
- Manage membership status.
- Profile image upload.
- Member search functionality.

Book Issue and Return

- Issue books to registered members.
- Return books with automatic status updates.
- Automatic overdue fine calculation.
- Fine payment tracking.
- Issue history maintenance.
- Overdue book monitoring.

Student and Teacher Portal

- Browse available books.
- Search books using multiple criteria.
- View issued books and due dates.
- Check pending fines.
- View newly added books.
- Update personal profile information.

Reports

- Book reports.
- Member reports.
- Issue and return reports.
- CSV export support.
- Dashboard analytics.

Settings

- Library configuration.
- Maximum book issue settings.
- Fine per day configuration.
- Maximum issue duration.
- Library information management.

Responsive User Interface

- Modern and user-friendly design.
- Mobile-responsive interface.
- Fast navigation.

Chapter-2 :- System Analysis

System analysis is the process of studying the existing system, identifying its limitations, understanding user requirements, and defining the functional and non-functional requirements of the proposed system. It provides a clear understanding of how the system will operate and helps in designing an efficient, reliable, and user-friendly application.

The proposed **Smart Library Management System** has been developed to automate library operations, improve the accuracy of record management, reduce manual work, and provide secure access to library resources. The system supports multiple users, including librarians, students, and teachers, through role-based authentication and centralized database management. By replacing traditional paper-based processes with a digital platform, the system enhances productivity, minimizes errors, and provides faster access to library information.

2.1 User Requirements (Software Requirement Specification - SRS)

The Software Requirement Specification (SRS) defines the functional and non-functional requirements of the Smart Library Management System. It specifies what the system should do and the quality attributes it must satisfy.

2.2.1 Functional Requirements

Administrator Module

The administrator (librarian) should be able to:

- Register and log in securely.
- Manage administrator profiles.
- Change passwords securely.
- Access the administrative dashboard.
- View real-time library statistics.
- Manage library settings.

Book Management Module

The system shall allow the administrator to:

- Add new books.
- Update book information.
- Delete books.
- Search books using title, author, ISBN, or category.
- Manage book categories.

- Upload book cover images.
- Track available book quantities.
- Maintain inventory information.

Member Management Module

The system shall allow the administrator to:

- Register students, teachers, and guest members.
- Update member information.
- Delete member records.
- Search members.
- Track membership status.
- Maintain member profiles.

Book Issue and Return Module

The system shall provide:

- Book issue functionality.
- Book return functionality.
- Due date management.
- Automatic overdue fine calculation.
- Fine payment tracking.
- Complete issue history.
- Overdue book monitoring.

Student and Teacher Portal

Registered members shall be able to:

- Log in securely.
- Browse available books.
- Search books.
- View issued books.
- View due dates.
- Check pending fines.
- Update profile information.

- View newly added books.

Reports Module

The system shall generate:

- Book reports.
- Member reports.
- Issue reports.
- Return reports.
- CSV export reports.
- Dashboard statistics.

2.2.2 Non-Functional Requirements

Performance Requirements

- Fast response time.
- Quick database retrieval.
- Efficient search functionality.
- Support for multiple users simultaneously.

Security Requirements

- Password encryption using bcrypt.
- Secure login authentication.
- Role-based access control.
- Session management.
- Input validation.
- Protection against unauthorized access.

Reliability Requirements

- Accurate data processing.
- Secure cloud database storage.
- Consistent system performance.
- Automatic error handling.

Usability Requirements

- User-friendly interface.

- Responsive design.
- Easy navigation.
- Minimal training required.

Scalability Requirements

The system should support:

- Increasing number of books.
- Increasing number of members.
- Large issue history.
- Future feature enhancements.

Maintainability Requirements

- Modular programming structure.
- Easy debugging.
- Simple code maintenance.
- Future software upgrades.

2.3 Hardware Requirements

The Smart Library Management System requires standard computer hardware for development, deployment, and execution. Since the application uses MongoDB Atlas as a cloud database, no dedicated database server is required locally.

Component Minimum Requirement Recommended Requirement

Processor	Intel Core i3 (6th Gen)	Intel Core i5/i7 (10th Gen or above)
RAM	4 GB	8 GB or higher
Storage	20 GB Free Disk Space	256 GB SSD or higher
Display	1366 × 768 Resolution	Full HD (1920 × 1080)
Internet	Broadband Connection	High-Speed Internet
Keyboard	Standard Keyboard	Standard Keyboard
Mouse	Optical Mouse	Optical Mouse

Development Hardware

- Processor: Intel Core i5 or above
- RAM: 8 GB or higher

- SSD Storage: 256 GB or higher
- Stable Internet Connection
- Laptop/Desktop Computer

Client Hardware

- Desktop Computer
- Laptop
- Tablet
- Smartphone with modern web browser

2.4 Software Requirements

The Smart Library Management System is developed using modern web technologies and software tools that provide security, scalability, and ease of maintenance.

Operating System

- Windows 10 / Windows 11
- Ubuntu Linux 20.04 or above
- macOS (optional)

Programming Languages :- Python 3.10+, JavaScript (Vanilla JavaScript)

Backend Framework:- Flask 2.3.x

Database:- MongoDB Atlas (Cloud NoSQL Database)

Database Driver:- Flask-PyMongo, PyMongo

Frontend Technologies :- HTML5, CSS3, JavaScript, Chart.js (Dashboard Analytics)

Development Tools:- Visual Studio Code, Git, GitHub, Postman, MongoDB Atlas Dashboard

Python Libraries:- Flask, Flask-PyMongo, Flask-CORS, Bcrypt, python-dotenv, Werkzeug

Web Browsers

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Server

- Flask Development Server
- Gunicorn (Production Deployment)

Chapter-3 :- System Design & Specification

The **System Design** of the Smart Library Management System defines the overall architecture, components, database structure, and interactions between users and the system. The application follows a three-tier architecture consisting of the presentation layer (HTML5, CSS3, and JavaScript), the application layer (Python Flask), and the data layer (MongoDB Atlas). The presentation layer provides an interactive and responsive user interface for librarians, students, and teachers, while the Flask backend processes user requests, performs business logic, validates data, and communicates securely with the MongoDB Atlas database. The system uses role-based authentication to ensure that administrators, students, and teachers can access only the functionalities assigned to their respective roles. This architecture improves security, maintainability, scalability, and overall system performance while allowing efficient management of books, members, issue records, returns, and reports.

The **System Specifications** describe the technologies and operational requirements used to develop and deploy the application. The Smart Library Management System is developed using **Python 3.10+** and the **Flask** framework, with **MongoDB Atlas** serving as the cloud-based NoSQL database for storing books, member records, issue transactions, categories, settings, and activity logs. The frontend is built using **HTML5, CSS3, and Vanilla JavaScript**, providing a responsive interface that is compatible with desktops, laptops, tablets, and smartphones. The system incorporates security mechanisms such as bcrypt password hashing, session management, CSRF protection, and input validation to safeguard user information. Additionally, it supports automated book issue and return processing, fine calculation, report generation, and real-time dashboard statistics, making it a reliable and efficient solution for modern library management in educational institutions.

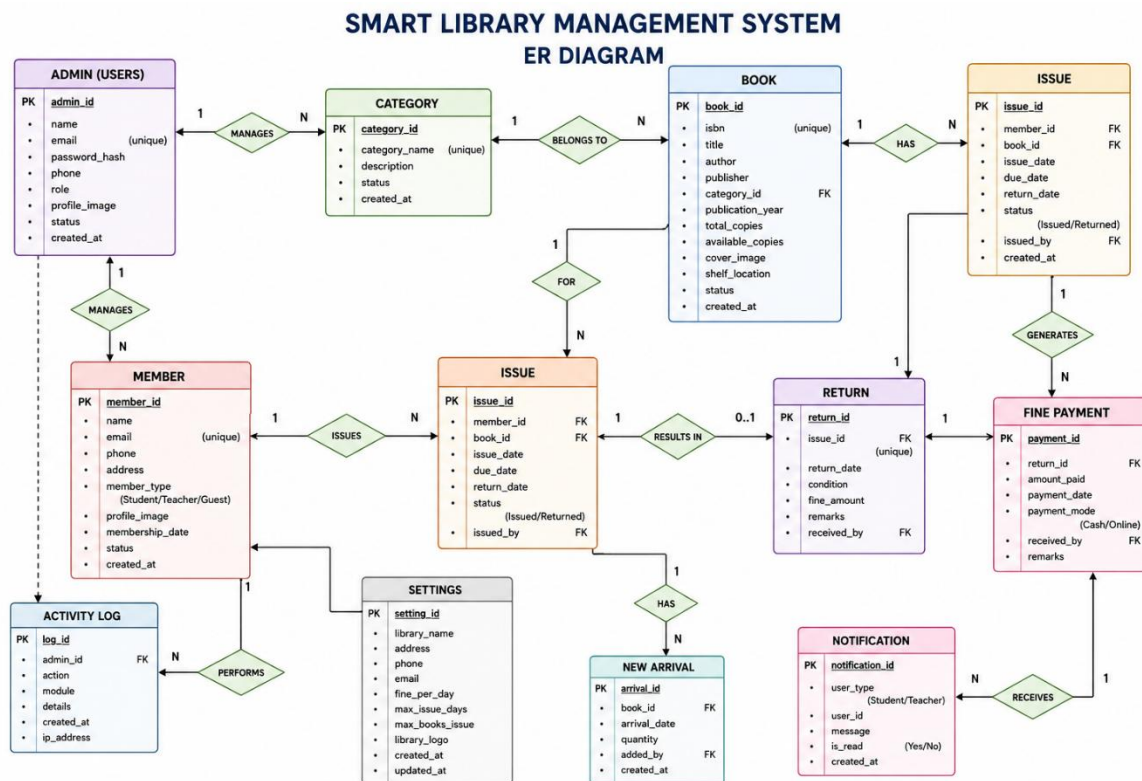
3.1 High Level Design :-

The **High Level Design (HLD)** describes the overall architecture and major components of the Smart Library Management System. It illustrates how different modules interact with each other and how data flows between users, the application, and the database. The system follows a **three-tier architecture**, consisting of the Presentation Layer, Application Layer, and Database Layer. The Presentation Layer provides an interactive web interface developed using HTML5, CSS3, and JavaScript, allowing librarians, students, and teachers to access the system through a web browser. The Application Layer is implemented using the Python Flask framework, which handles business logic, authentication, request processing, and communication between the user interface and the database. The Database Layer uses MongoDB Atlas to securely store information related to books, members, administrators, issue records, return records, reports, settings, and activity logs. This architecture ensures modularity, scalability, security, maintainability, and efficient performance.

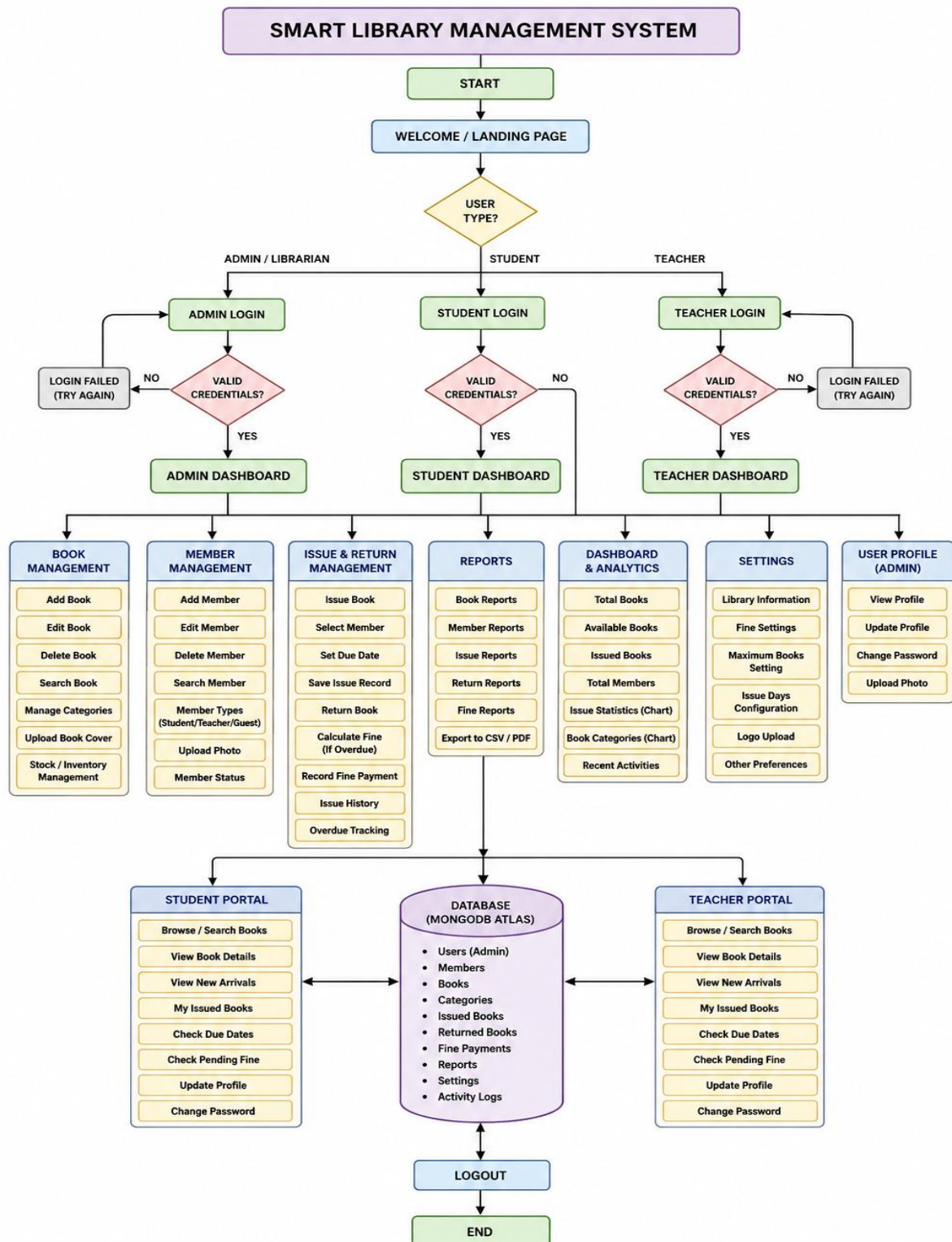
The HLD also defines the interaction between different system modules. Administrators can manage books, members, categories, issue and return transactions, reports, and system settings. Students and teachers can log in through their dedicated portal to search books, view issued books, check due dates and fines, and update their profiles. Each request made by a user is

processed by the Flask backend, validated for security, and then stored or retrieved from MongoDB Atlas. The system includes authentication, session management, automatic fine calculation, inventory tracking, report generation, and dashboard analytics. The modular design allows new features such as barcode scanning, QR code integration, email notifications, mobile applications, and AI-based book recommendations to be added in future versions without affecting the existing functionality.

3.1.1 Er-Diagram :-



3.1.2 Flowchart:-



3.2 Low Level Design :-

The **Low Level Design (LLD)** describes the internal implementation of each module in the Smart Library Management System. It explains how the system components interact, how individual functions are executed, and how data is processed within the application. The system is developed using a modular architecture, where each module performs a specific task and communicates with the database through the Flask backend. The frontend, developed using HTML5, CSS3, and JavaScript, collects user inputs and sends requests to the Flask server. The backend validates the requests, executes the required business logic, performs CRUD operations on MongoDB Atlas, and returns the appropriate response to the user.

The application consists of several modules, including Authentication, Book Management, Member Management, Category Management, Book Issue & Return, Fine Calculation, Reports, Dashboard, and Settings. Each module is implemented as an independent component, making the application easier to maintain, debug, and extend. The system also includes input validation, session management, password encryption using bcrypt, and role-based authorization to ensure secure access. This modular approach improves code reusability, scalability, and overall software quality.

3.2.1 Process Specification (Pseudo Code / Algorithm)

Algorithm 1: User Login

Purpose: Authenticate administrators, students, and teachers.

Pseudo Code

START

Display Login Page

Input Username and Password

IF Username or Password is empty THEN

 Display "Please enter all required fields"

ELSE

 Search user in database

 IF User exists THEN

 Verify encrypted password

 IF Password is correct THEN

 Create session

 Redirect to respective dashboard

 ELSE

```
        Display "Invalid Password"
    ENDIF
ELSE
    Display "User Not Found"
ENDIF
ENDIF
END
```

Algorithm 2: Add New Book

Purpose: Add a new book to the library database.

Pseudo Code

```
START
Open Add Book Form
Input:
    ISBN
    Title
    Author
    Publisher
    Category
    Quantity
Validate Input
IF Validation Successful THEN
    Check ISBN already exists
    IF ISBN not found THEN
        Save Book into MongoDB
        Display "Book Added Successfully"
    ELSE
        Display "ISBN Already Exists"
    ENDIF
ENDIF
```

ELSE

 Display Validation Error

ENDIF

END

Algorithm 3: Search Book

Purpose: Search books by Title, Author, ISBN, or Category.

Pseudo Code

START

Input Search Keyword

Search Database

IF Book Found THEN

 Display Book Details

ELSE

 Display "No Book Found"

ENDIF

END

Algorithm 4: Register Member

Purpose: Register a new student or teacher.

Pseudo Code

START

Enter Member Details

Validate Input

IF Email already exists THEN

 Display "Member Already Exists"

ELSE

 Save Member Record

 Generate Login Credentials

 Display Success Message

ENDIF

END

Algorithm 5: Issue Book

Purpose: Issue a book to a registered member.

Pseudo Code

START

Select Member

Select Book

Check Book Availability

IF Book Available THEN

 Create Issue Record

 Set Issue Date

 Set Due Date

 Reduce Available Quantity

 Save Transaction

 Display "Book Issued Successfully"

ELSE

 Display "Book Not Available"

ENDIF

END

Algorithm 6: Return Book

Purpose: Process the return of an issued book.

Pseudo Code

START

Select Issued Book

Calculate Delay Days

IF Delay Days > 0 THEN

$\text{Fine} = \text{Delay Days} \times \text{Fine Per Day}$

ELSE

 Fine = 0

ENDIF

Update Return Record

Increase Book Quantity

Save Transaction

Display Return Status

END

Algorithm 7: Fine Calculation

Purpose: Calculate overdue fine automatically.

Pseudo Code

START

Input Due Date

Input Return Date

Delay = Return Date – Due Date

IF Delay > 0 THEN

Fine = Delay × Fine_Per_Day

ELSE

Fine = 0

ENDIF

Return Fine Amount

END

Algorithm 8: Generate Reports

Purpose: Generate reports for library management.

Pseudo Code

START

Select Report Type

(Book Report /

Member Report /

Issue Report)

Retrieve Data from Database

Generate Report

Display Report

Export CSV if requested

END

Algorithm 9: Update Profile

Purpose: Allow users to update their profile.

Pseudo Code

START

Open Profile Page

Edit User Information

Validate Input

IF Validation Successful THEN

 Update Database

 Display "Profile Updated Successfully"

ELSE

 Display Validation Error

ENDIF

END

Algorithm 10: Logout

Purpose: Securely terminate the user session.

Pseudo Code

START

Destroy User Session

Clear Cookies

Redirect to Login Page

END

3.2.2 Screenshot

Create Account

Smart Library Management System

Register As

Student

Teacher

Librarian

Full Name

Phone Number

Department

Password

Min 6 chars, must include uppercase, lowercase, and numbers

Email

Username

Roll Number

Confirm Password

Profile Photo (Optional)

Choose File

No file chosen

Choose Image

Create Account

Already have an account? [Login here](#)

Library Portal

Login to access your library account

Student

Teacher

Librarian

Username or Email

Password

Login

Don't have an account? [Register here](#)

Are you an admin? [Admin Login](#)

Library Portal

Student

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Satyjit Sahoo

Welcome, Satyjit Sahoo!

Welcome to your Student library account

Total Books
4

Available Books
4

Books With Me
1

New Arrivals
4

Your Pending Fines

₹95

Please return overdue books to avoid further fines.

My Recent Books

Python

Due: 18 Aug 2026 | Status: issued

Library Portal

Student

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Satyjit Sahoo

Browse Books

Search and explore the library collection

All Categories

Python

Rashmi Ranjan Sahoo

9 available

Algebra

Subhrajit das

29 available

Spoken english

Rashmi Ranjan das

30 available

home Science

Satyjit Sahoo

49 available

Library Portal

Student

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Search books...

Satyjit Sahoo

My Issued Books

Track your issued books, due dates, and fines

All Records

Book	Issue Date	Due Date	Returned On	Status	Fine
Python Rashmi Ranjan Sahoo	1 Aug 2026	15 Aug 2026	20 Aug 2026	Returned	₹25
Python Rashmi Ranjan Sahoo	2 Aug 2026	16 Aug 2026	4 Sept 2026	Returned	₹95
Spoken english Rashmi Ranjan das	2 Aug 2026	12 Aug 2026	4 Sept 2026	Returned	₹115
Python Rashmi Ranjan Sahoo	4 Aug 2026	18 Aug 2026	-	Issued	₹0

Library Portal

Student

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Search books...

Satyjit Sahoo

Pay Fines

Pay your library fines online securely

Outstanding Fines

₹95

Book	Reason	Due Date	Fine Amount	Action
Python	Overdue book return	16 Aug 2026	₹95	Pay Now

Payment History

Payment ID	Transaction ID	Amount	Method	Date	Status
PAY9C9B322F	TXN888261782	₹25	UPI	1 Aug 2026	Confirmed
PAYC715D026	TXN140902836	₹115	UPI	2 Aug 2026	Confirmed

Library Portal

Student

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Search books...

Satyjit Sahoo

My Returned Books

View your returned books, return dates, and fines

Search returned books...

Book	Issue Date	Due Date	Returned On	Status	Fine Amount
Python Rashmi Ranjan Sahoo	1 Aug 2026	15 Aug 2026	20 Aug 2026	Returned	₹25
Python Rashmi Ranjan Sahoo	2 Aug 2026	16 Aug 2026	4 Sept 2026	Returned	₹95
Spoken english Rashmi Ranjan das	2 Aug 2026	12 Aug 2026	4 Sept 2026	Returned	₹115

Smart Library

Order ID:

ORD232357386

Customer:

Satyjit Sahoo

Amount:

₹95.00

UPI

Card

Wallet

Net Banking

Select UPI App

G

Google Pay

P

PhonePe

P

Paytm

B

BHIM

A

Amazon Pay

Enter UPI ID

UPI

Card

Wallet

Net Banking

Select UPI App

G

Google Pay

P

PhonePe

P

Paytm

B

BHIM

A

Amazon Pay

Enter UPI ID

9348698773@jbl

Example: abc@ybl, abc@paytm, abc@oksbi

Verify UPI

Pay Now

✓

Verified Successfully

✓

Payment Successful

Transaction Successful

Transaction ID:

TXN312693295

Order ID:

ORD232357386

Amount Paid:

₹95.00

Date:

4/8/2026

Time:

7:20:46 am

Download Receipt

Continue

Library Portal

Teacher

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Search books...

Rashmi Ranjan Sahoo

Welcome, Rashmi Ranjan Sahoo!

Welcome to your Teacher library account

Total Books4

Available Books4

Books With Me1

New Arrivals4

Your Pending Fines

₹65

Please return overdue books to avoid further fines.

My Recent Books

home Science

Due: 18 Aug 2026 | Status: issued

Library Portal

Teacher

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Search books...

Rashmi Ranjan Sahoo

Browse Books

Search and explore the library collection

Search by title, author, ISBN...

All Categories

Python

Rashmi Ranjan Sahoo

9 available

Algebra

Subhrajit das

29 available

Spoken english

Rashmi Ranjan das

30 available

home Science

Satyjit Sahoo

49 available

Library Portal

Teacher

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Search books...

Rashmi Ranjan Sahoo

My Issued Books

Track your issued books, due dates, and fines

All Records

Book	Issue Date	Due Date	Returned On	Status	Fine
Algebra Subhrajit das	1 Aug 2026	11 Aug 2026	30 Aug 2026	Returned	₹95
Spoken english Rashmi Ranjan das	1 Aug 2026	15 Aug 2026	31 Aug 2026	Returned	₹80
home Science Satyjit Sahoo	2 Aug 2026	16 Aug 2026	3 Sept 2026	Returned	₹90
Spoken english Rashmi Ranjan das	4 Aug 2026	18 Aug 2026	31 Aug 2026	Returned	₹65
home Science Satyjit Sahoo	4 Aug 2026	18 Aug 2026	-	Issued	₹0

Library Portal

Teacher

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Search books...

Rashmi Ranjan Sahoo

My Returned Books

View your returned books, return dates, and fines

Search returned books...

Book	Issue Date	Due Date	Returned On	Status	Fine Amount
Algebra Subhrajit das	1 Aug 2026	11 Aug 2026	30 Aug 2026	Returned	₹95
Spoken english Rashmi Ranjan das	1 Aug 2026	15 Aug 2026	31 Aug 2026	Returned	₹80
home Science Satyjit Sahoo	2 Aug 2026	16 Aug 2026	3 Sept 2026	Returned	₹90
Spoken english Rashmi Ranjan das	4 Aug 2026	18 Aug 2026	31 Aug 2026	Returned	₹65

Library Portal

Teacher

Dashboard

Browse Books

My Issued Books

My Returned Books

Pay Fines

New Arrivals

My Profile

Logout

Search books...

Rashmi Ranjan Sahoo

Pay Fines

Pay your library fines online securely

Outstanding Fines

₹65

Book	Reason	Due Date	Fine Amount	Action
Spoken english	Overdue book return	18 Aug 2026	₹65	Pay Now

Payment History

Payment ID	Transaction ID	Amount	Method	Date	Status
PAY1D30A65B	TXN242713209	₹95	Net Banking - State Bank of India (SBI)	1 Aug 2026	Confirmed
PAYFF48F719	TXN580618046	₹80	UPI	1 Aug 2026	Confirmed
PAYF5836249	TXN832279062	₹90	Wallet - Paytm Wallet	2 Aug 2026	Confirmed

Smart Library

Dashboard

Books

Members

Issue Book

Return Book

Payment Received

Reports

Profile

Settings

Search books, members...

Admin

Book Management

Categories

Add Book

Search by title, author, ISBN...

All Categories

Book ID	Title	Author	Category	ISBN	Quantity	Available	Price	Actions
BKEF6F9181	Python	Rashmi Ranjan Sahoo	Computer Science	0000000001	10		₹150	Edit Delete
BK2446C8BC	Algebra	Subhrajit das	Mathematics	000000000002	30		₹200	Edit Delete
BKF8177B0D	Spoken english	Rashmi Ranjan das	English	0000000000003	30		₹200	Edit Delete
BK869CB299	home Science	Satyjit Sahoo	Physiscs	0123456789012	50		₹300	Edit Delete

Smart Library

Dashboard

Books

Members

Issue Book

Return Book

Payment Received

Reports

Profile

Settings

Logout

Search books, members

Admin

Dashboard

Welcome to Smart Library Management System

Total Books

4

Available Books

4

Issued Books

3

Total Members

4

Issue Statistics

Issued

Returned

Overdue

Book Categories

Computer Science

English

Mathematics

Physics

0

0.1

0.2

0.3

0.4

0.5

0.6

0.7

0.8

0.9

1.0

Quick Actions

Issue Book

Return Book

Add Book

Add Member

Recent Activities

Book issued - Member: Rashmi Ranjan Sahoo, Book: home Science

2026-08-04 01:42:45

Book issued - Member: Satyjit Sahoo, Book: Python

2026-08-04 01:42:30

Book returned - Issue: 6a7142d5bc08c1cd1ca61e4a, Fine: ₹65

2026-08-04 01:40:17

Book issued - Member: Rashmi Ranjan Sahoo, Book: Spoken english

2026-08-04 01:39:34

Book returned - Issue: 6a6e1c2591fab4ef92ebf144, Fine: ₹95

2026-08-04 01:39:12

Book 6a6df8689bd6811780b8e7f5 updated

2026-08-02 11:28:59

Fine payment confirmed - Rashmi Ranjan Sahoo, ₹90.0

2026-08-02 11:25:27

Fine payment confirmed - Satyjit Sahoo, ₹115.0

2026-08-02 11:25:13

Book returned - Issue: 6a6f28150736a55a67f90e22, Fine: ₹90

2026-08-02 11:21:39

Book returned - Issue: 6a6f280a0736a55a67f90e20, Fine: ₹115

2026-08-02 11:21:27

Search books, members...

Admin

Return Book

Select Issued Book to Return

Choose Issue Record

Return Date

04-08-2026

Fine Amount

₹0

Return Book

Return Book History

Search by member, book, issue ID...

Issue ID	Member	Book	Issue Date	Due Date	Returned On	Fine Amount	Status
ISS2379D1A1	Unknown	Python 0000000001	1/8/2026	15/8/2026	15/8/2026	₹0	Returned
ISS23816416	Unknown	Algebra 0000000000002	1/8/2026	11/8/2026	1/8/2026	₹0	Returned



Search books, members...



Admin

Issue Book

Select Member

Choose Member

Select Book

Choose Book

Issue Date

04-08-2026



Issue Days

14

✓ Issue Book

Recently Issued Books

Issue ID	Member	Book	Issue Date	Return Date	Status
ISS2379D1A1	Unknown	Python	1/8/2026	15/8/2026	Returned
ISS23816416	Unknown	Algebra	1/8/2026	11/8/2026	Returned
ISSA1CF38CA	Satyjit Sahoo	Python	1/8/2026	15/8/2026	Returned
ISS36DB07AF	Unknown	Algebra	1/8/2026	2/8/2026	Overdue
ISSD9E80D7A	Rashmi Ranjan Sahoo	Algebra	1/8/2026	11/8/2026	Returned
ISS54BA5E2E	Rashmi Ranjan Sahoo	Spoken english	1/8/2026	15/8/2026	Returned
ISS521D6477	Satyjit Sahoo	Python	2/8/2026	16/8/2026	Returned
ISS85F48257	Satyjit Sahoo	Spoken english	2/8/2026	12/8/2026	Returned
ISSF28E527E	Rashmi Ranjan Sahoo	home Science	2/8/2026	16/8/2026	Returned
ISS3C9E5ECD	Rashmi Ranjan Sahoo	Spoken english	4/8/2026	18/8/2026	Returned



Search books, members...



Admin

Payments Received

Online fine payments from students & teachers. Confirm payments to clear fines.



Pending

2



Confirmed

5



Total Collected

₹405



Pending Amount

₹160

Search by name, payment ID, transaction ID...

All Status

Payment ID	Member	Type	Books	Amount	Method	Date	Status	Actions
PAY9C9B322F	Satyjit Sahoo MEMB4B6793C	Student	Python	₹25	UPI	1 Aug 2026	Confirmed	Done
PAY1D30A65B	Rashmi Ranjan Sahoo MEM4EAE81C9	Teacher	Algebra	₹95	Net Banking - State Bank of India (SBI)	1 Aug 2026	Confirmed	Done
PAYFF48F719	Rashmi Ranjan Sahoo MEM4EAE81C9	Teacher	Spoken english	₹80	UPI	1 Aug 2026	Confirmed	Done
PAYC715D026	Satyjit Sahoo MEMB4B6793C	Student	Spoken english	₹115	UPI	2 Aug 2026	Confirmed	Done
PAYF5B36249	Rashmi Ranjan Sahoo MEM4EAE81C9	Teacher	home Science	₹90	Wallet - Paytm Wallet	2 Aug 2026	Confirmed	Done
PAYBE019A25	Satyjit Sahoo MEMB4B6793C	Student	Python	₹95	UPI	4 Aug 2026	Pending	✓ Confirm
PAY4591E56C	Rashmi Ranjan Sahoo MEM4EAE81C9	Teacher	Spoken english	₹65	UPI	4 Aug 2026	Pending	✓ Confirm

Reports

Book Report

View all books inventory

Generate Report

Member Report

View all members information

Generate Report

Issue Report

View issue and return statistics

Generate Report

Export Data

Export reports as CSV

Books

Export CSV

Settings

Library Information

Library Name

Smart Library

Address

GIETU

Working Hours

9 AM - 5 PM

Fine Per Day (₹)

5

Max Books to Issue

5

Default Issue Days

Save Settings

My Profile



Change Photo

Full Name

Admin

Email

admin@library.com

Phone

9999999999

Username

admin

Update Profile

Change Password

Current Password

New Password

Confirm New Password

Change Password

Chapter-4 :- Methodology Section

The methodology describes the systematic approach followed during the design, development, testing, and deployment of the Smart Library Management System. A structured Software Development Life Cycle (SDLC) methodology was adopted to ensure that the project was completed in an organized and efficient manner. The development process involved requirement gathering, system analysis, system design, implementation, testing, deployment, and maintenance. Each phase was carefully planned and executed to develop a secure, reliable, and user-friendly web application that automates library operations. The application was built using **Python Flask** as the backend framework, **MongoDB Atlas** as the cloud-based database, and **HTML5, CSS3, and Vanilla JavaScript** as the frontend technologies. This methodology ensured modular development, easier debugging, improved maintainability, and future scalability.

The project follows a client-server architecture where users interact with the web interface through a browser. User requests are processed by the Flask server, validated for correctness and security, and then stored or retrieved from MongoDB Atlas. The system implements role-based authentication to provide separate access for librarians, students, and teachers. Passwords are encrypted using bcrypt, while session management and server-side validation improve application security. Throughout development, individual modules such as Authentication, Book Management, Member Management, Book Issue & Return, Reports, Dashboard, and Settings were developed and tested independently before integrating them into the complete system. This modular methodology reduced development complexity and improved software quality.

4.1 Approach Used

The Smart Library Management System was developed using an **Agile Software Development Approach**. Agile enables the application to be developed in small, manageable modules, allowing continuous testing, debugging, and improvement throughout the development lifecycle. Instead of building the entire system at once, each module was designed, implemented, tested, and integrated sequentially. This approach increased development efficiency and ensured that every module functioned correctly before moving to the next stage.

The development process followed the steps below:

Step 1: Requirement Analysis

- Identify problems in the existing manual library system.
- Collect functional and non-functional requirements.
- Define user roles (Administrator, Student, Teacher).

Step 2: System Analysis

- Study existing library operations.
- Prepare Software Requirement Specification (SRS).

- Identify hardware and software requirements.

Step 3: System Design

- Design High-Level Design (HLD).
- Design Low-Level Design (LLD).
- Create database schema.
- Prepare ER Diagram, Flowchart, and Structure Chart.

Step 4: Database Design

- Design MongoDB collections.
- Define relationships among books, members, administrators, issues, returns, reports, and settings.

Step 5: Frontend Development

- Develop responsive user interfaces using HTML5, CSS3, and JavaScript.
- Create dashboards for administrators, students, and teachers.

Step 6: Backend Development

- Implement RESTful APIs using Python Flask.
- Develop authentication, CRUD operations, fine calculation, and reporting modules.
- Connect the application with MongoDB Atlas.

Step 7: Testing

- Perform Unit Testing.
- Perform Integration Testing.
- Perform System Testing.
- Fix identified errors and validate functionality.

Step 8: Deployment and Maintenance

- Deploy the application on a web server.
- Monitor system performance.
- Implement future updates and enhancements.

Chapter-5 :- Implementation

The implementation phase successfully transformed the proposed Smart Library Management System into a fully functional web-based application. The system was developed using **Python Flask** for the backend, **MongoDB Atlas** for cloud-based database management, and **HTML5**, **CSS3**, and **Vanilla JavaScript** for the frontend. A modular implementation approach was adopted, where each module—including Authentication, Dashboard, Book Management, Member Management, Book Issue and Return, Fine Management, Reports, and Settings—was developed, tested, and integrated independently. This approach improved code organization, maintainability, scalability, and overall software quality while ensuring that each module functioned correctly before integration.

The implemented system provides secure role-based authentication for librarians, students, and teachers, enabling efficient management of library operations through an intuitive and responsive interface. Core functionalities such as book inventory management, member registration, automated issue and return processing, overdue fine calculation, report generation, and profile management were successfully integrated with the MongoDB Atlas database. Security features including bcrypt password hashing, session management, input validation, and access control were implemented to protect user data and ensure reliable system operation. Overall, the implementation satisfies the functional and non-functional requirements identified during system analysis and delivers a secure, scalable, and efficient library management solution that can be further enhanced with features such as barcode scanning, QR code integration, mobile applications, and online book reservation.

Chapter-6 :- Testing

Testing is one of the most important phases of the Software Development Life Cycle (SDLC). It is performed to verify that the Smart Library Management System functions correctly, meets the specified requirements, and provides reliable and secure services to users. The objective of testing is to identify and eliminate errors, validate system functionality, ensure data integrity, and confirm that the application performs efficiently under different conditions. Various testing techniques were applied during the development process to ensure that every module operates as expected and integrates seamlessly with the overall system.

The Smart Library Management System was tested using multiple testing methods, including Unit Testing, Integration Testing, System Testing, Functional Testing, Validation Testing, Security Testing, Performance Testing, User Acceptance Testing (UAT), and

Compatibility Testing. Each module, such as Authentication, Book Management, Member Management, Book Issue & Return, Fine Calculation, Dashboard, Reports, and Settings, was tested individually before being integrated into the complete application. The testing process confirmed that the system performs accurately, securely, and efficiently while satisfying all functional and non-functional requirements.

6.1 Objectives of Testing

The objectives of testing are:

- To verify that all system modules function correctly.
- To identify and eliminate software defects.
- To ensure accurate database operations.
- To validate user inputs.
- To verify secure authentication and authorization.
- To ensure correct book issue and return operations.
- To validate automatic fine calculation.
- To ensure compatibility across different web browsers.
- To evaluate overall system performance and reliability.

6.2 Types of Testing

6.2.1 Unit Testing

Unit Testing verifies the functionality of individual modules independently before integration.

Modules Tested:

- Authentication Module
- Book Management Module
- Member Management Module

- Book Issue Module
- Return Module
- Fine Calculation Module
- Reports Module
- Settings Module

Result: All individual modules executed successfully and produced the expected output.

6.2.2 Integration Testing

Integration Testing verifies communication between different modules.

Integration Tested:

- Login → Dashboard
- Dashboard → Book Management
- Book Management → MongoDB Atlas
- Member Management → Issue Module
- Issue Module → Return Module
- Return Module → Fine Calculation
- Reports → Database

Result: All integrated modules communicated successfully without data inconsistency.

6.2.3 System Testing

System Testing evaluates the complete Smart Library Management System in an integrated environment.

Activities Performed:

- Administrator Login

- Student Login
- Teacher Login
- Book CRUD Operations
- Member CRUD Operations
- Book Issue
- Book Return
- Fine Calculation
- Dashboard Statistics
- Report Generation

Result: The complete application functioned according to the project requirements.

6.2.4 Functional Testing

Functional Testing verifies that every feature performs according to its specification.

Functions Tested:

- User Registration
- User Login
- Book Search
- Add Book
- Update Book
- Delete Book
- Member Registration
- Issue Book
- Return Book

- Report Generation
- Profile Update

Result: All functions operated correctly.

6.2.5 Validation Testing

Validation Testing ensures that user inputs satisfy predefined validation rules.

Validation Checked:

- Required Fields
- Valid Email Format
- ISBN Validation
- Password Strength
- Mobile Number Format
- Duplicate Book Entries
- Duplicate Member Records

Result: Invalid inputs were rejected successfully.

6.2.6 Security Testing

Security Testing verifies that user information and application data remain protected.

Security Features Tested:

- Password Encryption
- Secure Login
- Session Management
- Role-Based Access
- Unauthorized Access Prevention

- Input Validation

Result: Security mechanisms functioned successfully.

6.2.7 Performance Testing

Performance Testing measures application responsiveness under normal workload.

Parameters Tested:

- Login Response Time
- Book Search Speed
- Dashboard Loading
- Database Query Performance
- Report Generation Time

Result: The application performed efficiently with minimal response time.

6.2.8 Compatibility Testing

Compatibility Testing ensures that the application operates correctly across different browsers and devices.

Browsers Tested:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Devices Tested:

- Desktop
- Laptop
- Tablet

- Smartphone

Result: The application displayed correctly on all tested platforms.

6.2.9 User Acceptance Testing (UAT)

User Acceptance Testing confirms that the system satisfies end-user requirements.

Users Involved:

- Librarian
- Student
- Teacher

Result: Users successfully completed all required tasks, and the system met their expectations.

6.2.10 Test Cases

Test Case ID	Test Scenario	Input	Expected Output	Actual Output	Status	Remedial Action
TC-01	Administrator Login	Valid Username & Password	Login Successful	Login Successful	Pass	Not Required
TC-02	Invalid Login	Wrong Password	Error Message	Error Displayed	Pass	Improved password validation
TC-03	Add Book	Valid Book Details	Book Added	Book Added	Pass	Not Required

			Successfully	Successfully		
TC-04	Duplicate ISBN	Existing ISBN	Duplicate ISBN Error	Error Displayed	Pass	Duplicate validation implemented
TC-05	Search Book	Book Title	Display Book Details	Book Found	Pass	Not Required
TC-06	Register Member	Valid Member Details	Member Registered	Member Registered	Pass	Not Required
TC-07	Duplicate Member Email	Existing Email	Registration Failed	Duplicate Email Error	Pass	Email uniqueness enforced
TC-08	Issue Book	Available Book	Book Issued Successfully	Book Issued Successfully	Pass	Inventory updated automatically
TC-09	Return Book	Returned After Due Date	Fine Calculated	Fine Calculated Correctly	Pass	Automatic fine calculation verified
TC-10	Generate Report	Book Report Request	Report Generated	Report Generated Successfully	Pass	CSV export verified

TC- 11	Update Profile	Valid User Information	Profile Updated	Profile Updated Successfully	Pass	Input validation improved
TC- 12	Logout	Logout Request	Session Destroyed	Logout Successful	Pass	Session cleared successfully

Chapter-7 :- Conclusion & Future Scope.

7.1 Conclusion

The **Smart Library Management System** has been successfully designed, developed, and implemented as a comprehensive web-based application for automating library operations. The project effectively addresses the limitations of traditional manual library management by providing a centralized, secure, and user-friendly platform for managing books, members, issue and return transactions, overdue fine calculations, reports, and library settings. The system simplifies the daily responsibilities of librarians while offering convenient self-service features for students and teachers through dedicated user portals.

The application was developed using **Python Flask** as the backend framework, **MongoDB Atlas** as the cloud-based database, and **HTML5**, **CSS3**, and **Vanilla JavaScript** for the frontend. By following a structured Software Development Life Cycle (SDLC), the project successfully completed all phases, including requirement analysis, system design, implementation, testing, and deployment. Comprehensive testing confirmed that all modules operate accurately and reliably, ensuring secure authentication, efficient database management, responsive user interfaces, and automated library workflows.

The implemented system improves operational efficiency by reducing paperwork, minimizing human errors, maintaining accurate records, and providing faster access to library information. Features such as role-based authentication, automated fine calculation, inventory tracking, dashboard analytics, report generation, and cloud-based data storage contribute to a modern and efficient library management solution. The modular architecture also ensures that the application is scalable, maintainable, and adaptable to future technological advancements.

In conclusion, the Smart Library Management System successfully achieves its primary objective of providing a secure, reliable, and efficient digital solution for library automation. The project demonstrates the practical application of full-stack web development, database management, software engineering principles, and secure web technologies. It can be effectively deployed in schools, colleges, universities, and other educational institutions to improve library administration and enhance the overall user experience.

7.2 Future Scope

Although the Smart Library Management System meets the current requirements of library automation, several enhancements can be incorporated in future versions to improve functionality, user experience, and overall system capabilities.

Future Enhancements

1. Barcode and QR Code Integration

- Implement barcode and QR code scanning for faster book issue, return, and inventory management.

2. Online Book Reservation

- Allow members to reserve books online before visiting the library.

3. Book Renewal System

- Enable students and teachers to renew issued books through the online portal without visiting the library.

4. Email and SMS Notifications

- Send automatic notifications for due dates, overdue books, successful issue/return transactions, and library announcements.

5. Mobile Application

- Develop Android and iOS mobile applications for convenient access to library services.

6. Digital Library Integration

- Integrate e-books, digital journals, research papers, and online learning resources into the system.

7. AI-Based Book Recommendation

- Implement an intelligent recommendation system that suggests books based on users' reading history, interests, and borrowing patterns.

8. Advanced Analytics Dashboard

- Include detailed analytical reports, borrowing trends, most popular books, member activity statistics, and predictive insights for better decision-making.

9. Online Fine Payment

- Integrate secure online payment gateways to allow members to pay overdue fines digitally.

10. Multi-Language Support

- Provide multilingual interfaces to make the system accessible to users from different linguistic backgrounds.

11. **Cloud Backup and Disaster Recovery**

- Implement automated cloud backup and recovery mechanisms to ensure data safety and business continuity.

12. **RFID-Based Library Automation**

- Integrate RFID technology for automated book identification, inventory tracking, and self-service book issue and return.

13. **Real-Time Notifications**

- Provide instant notifications for new book arrivals, reservation status, account updates, and important library events.

14. **Biometric Authentication**

- Introduce fingerprint or facial recognition for secure administrator and member authentication.

15. **Integration with Institutional Systems**

- Connect the library management system with college ERP, student information systems, and academic management platforms for seamless data sharing.